JFEM and OpenLUDWIG

A structural FE solver and a GPU-native CFD solver, both pure Julia, both ready for industrial workflows

Raul – BSC ARELIS

May 27, 2026

In One Slide

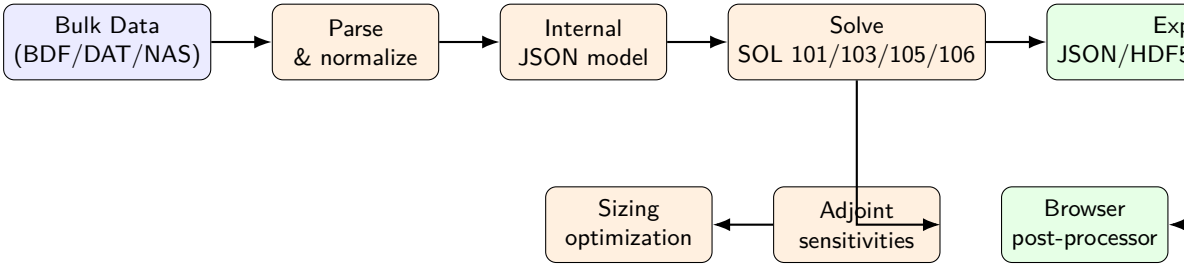
- **JFEM** – a Julia structural finite-element solver, input-compatible with Nastran. SOL 101, 103, 105, experimental 106. Sufficient parity with Nastran shell buckling for industrial-grade studies.
- **OpenLUDWIG** – a fully compressible 3D Lattice-Boltzmann CFD solver, GPU-native, written from scratch in Julia. Aims at state of the art performance per watt.
- Both pure Julia. Both with a JavaScript front-end powered by Babylon.js. Both exchange data with the client via msgpack with binary JSON payloads.
- Both released under AGPLv3 on GitHub, developed at the Barcelona Supercomputing Centre under the **ARELIS** project.
- Two tiers: a **public sub-industrial** tier always on GitHub, and a **private tier** tailored to our processes.

JFEM – What It Is

A structural finite-element solver written in pure Julia, designed to accept the same input language as Nastran (BDF / DAT / NAS bulk data).

- Three-stage workflow: **parse** BDF → build internal model → **solve** sparse system → **export** (JSON, HDF5, VTK, custom `.jfem`).
- Solution sequences: **SOL 101** (linear static), **SOL 103** (normal modes, with SOL 63 normalized), **SOL 105** (linear buckling), experimental **SOL 106** (geometrically nonlinear).
- Elements: shell, bar, beam, rod, spring, rigid, mass, solid; composite laminates via `PCOMP`.
- Adjoint sensitivity framework for static responses and buckling eigenvalues; sizing optimization driver covers shell thickness and bar area.

JFEM – Pipeline



Plain Julia code, no compiled C extensions in the solver core.

JFEM – How It Got Here

Development history – agentic from day one:

1. Started **from scratch**, no reference solver, just the BDF spec.
2. **MYSTRAN** (open Nastran-like) introduced as cross-validation reference.
3. **TACS** (modern adjoint-capable solver) added as second reference; the solver was made "aware" of TACS conventions.
4. Significant technical literature fed to the agent to derive element formulations correctly.
5. Today: Nastran is sufficiently well replicated, particularly for **shell buckling**, to drive real design studies.

JFEM – CQUAD4 Parity, A Story In Itself

The CQUAD4 shell element is Nastran's workhorse. Achieving parity exposed a lot of hidden machinery:

- multiple kernel variants (standard MITC, explicit-covariant, auto-covariant, exact-membrane);
- branch-aware adjoint linearizations on each subset;
- residual / Gauss-point / blended prestress-field selectors frozen during basis probing;
- mode-tracked finite-difference fallback for the hardest curved-shell groups;
- MAC-based regressions on barrel, dome, and tapered-cylinder families.

Opportunity: this opens the door to *modernizing* the formulation – using a TACS or Abaqus-style kernel where it improves accuracy, without losing Nastran input compatibility.

JFEM vs. TACS vs. MYSTRAN – Capability Snapshot

	JFEM	**TACS**	**MYSTRAN**
Language	Julia	C++ with Python bindings	FORTRAN
Input	Nastran BDF (broad subset)	Native; BDF import limited	Nastran BDF (large subset)
SOL 101 / 103 / 105	Yes	Yes	Yes
SOL 106 (NL static)	Experimental	Yes (arc-length)	Limited
Composites (PCOMP)	Yes	Yes (extensive)	Partial
Adjoint sensitivities	Static and buckling	Full and mature	No
Auto-differentiation	ForwardDiff on selected paths	Native AD-aware	No
GPU readiness	CPU only	Mostly CPU	CPU
License	AGPLv3	Apache 2.0	Public domain / open

JFEM's near-term goal is to match TACS on adjoint coverage and to add an arc-length nonlinear branch.

JFEM Roadmap

- **SOL 106 arc-length** solver with a TACS-style (or, preferably, Abaqus-style) formulation – nonlinear static with snap-through and post-buckling handling.
- **Aeroelastic coupling** via AVL or `VortexLattice.jl` for first-pass static aeroelasticity.
- **Full adjoint coverage** and automatic differentiation of all relevant properties (geometry, material, layup, sizing).
- Tighter integration with **OpenLUDWIG** to consume CFD-derived nodal loads.
- Continued CQUAD4 modernization track.

JFEM vs. Nastran – Wall-Clock Timing

Illustration to insert:
Bar chart: SOL 105 wall-clock for representative decks (small, mid, large) comparing Nastran 70.5 and JFEM on the same hardware. Indicate threads and sysimage status.

Headline: JFEM with the fast sysimage path is competitive for small and mid decks, dominated by Nastran on the largest decks where MUMPS-style factorization wins. Active work targets that gap.

OpenLUDWIG – What It Is

A fully compressible, GPU-native 3D Lattice-Boltzmann CFD solver.

- Pure Julia, with CUDA kernels via Julia's GPU stack.
- Compressible LBM (not just incompressible Bhatnagar–Gross–Krook).
- Targets state-of-the-art performance and accuracy.
- Developed **from scratch** agentically; once the development plateaued, the repositories of **OpenLB** and **PALABOS** were fed to the agent as reference, along with significant literature.

OpenLUDWIG – Why Lattice Boltzmann?

- **Local** collision-and-stream updates: maps cleanly to GPU threads and gives near-linear scaling.
- No global Poisson solve: avoids the largest serial bottleneck in incompressible CFD.
- Simple boundary handling for moving and complex geometry.
- Fully compressible variant unlocks aerodynamic regimes that the classical incompressible LBM cannot touch.

Illustration to insert:
Visualization: OpenLUDWIG flow field around a generic transport wing-body, browser post-processor render with velocity iso-surfaces.

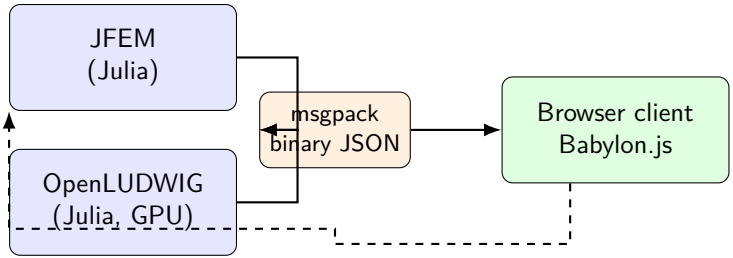
OpenLUDWIG → JFEM Coupling

OpenLUDWIG can compute **nodal loads** on a finite-element mesh directly from the CFD pressure and shear fields:

- One LBM solution → one set of nodal loads per flight condition.
- Auto-generated FE models can be loaded straight from the LBM result.
- **Inertial loads** (gravity, linear and rotational accelerations) are added on the FE side to allow **free-free** support: the structure flies in equilibrium to machine precision before any deformation.

This is the path to aeroelastic studies without needing a commercial fluid-structure coupling stack.

Architecture – Julia Server, JavaScript Client



Same client code can drive both solvers. The transport is intentionally thin: msgpack + binary JSON, no REST gymnastics.

JavaScript Post-Processor

Common post-processor for JFEM and OpenLUDWIG, written in pure JavaScript.

- Babylon.js as 3D engine (WebGPU when available, WebGL fallback).
- Reads the custom `.jfem` binary, plus VTK and HDF5 via dedicated loaders.
- Field overlays, deformation animation, mode-shape inspection, buckling-eigenvector cycling.
- Slicing, clipping, probe queries, screenshot capture (used in turn for agentic debugging – the agent reads the screenshot back).

Illustration to insert:

Screenshot: JFEM browser post-processor with CQUAD4 deformed shape and von Mises overlay on a curved panel.

Why Julia For Both Solvers

- **Native performance** – LLVM JIT, type stability, SIMD; no two-language barrier between glue and inner loops.
- **First-class GPU support** – CUDA.jl, AMDGPU.jl, oneAPI.jl all expose the same array abstraction.
- **Composable AD** – ForwardDiff, Zygote, Enzyme work on generic code, not on a separate IR.
- **Package ecosystem** – HDF5, KrylovKit, IterativeSolvers, AlgebraicMultigrid, WriteVTK; no need to roll our own.
- **Agent-friendly** – legible code, REPL-driven workflows, and no header/source split to keep in sync.

Licensing And Distribution

- Both solvers under **AGPLv3** on GitHub.
- Developed at the **Barcelona Supercomputing Centre** under the **ARELIS** project.
- Two tiers:
  - **Public tier** – always available on GitHub, providing "sub-industrial" but useful capabilities.
  - **Private tier** – tailored to internal processes and customer-specific workflows.
- Validation evidence (decks, run reports, registry) lives in a private development workspace; only finished PDFs and code reach GitHub.

Take-Aways

- A small team can stand up two serious solvers in a year by combining **agentic development** with a disciplined workspace and a tight reference set.
- Julia removes the two-language tax and is the right host language for both CPU FE work and GPU CFD work.
- Nastran-compatible input + a modern open core is a viable modernization path for industrial structural analysis.
- GPU-native LBM CFD makes per-flight-condition load generation tractable on workstation-class hardware.
- The whole stack is reproducible, documented, and queryable – by humans and by other AI agents.

Questions

Thank you.

Questions, and ideas for use cases we should attack next.